Motion-Gated Dynamic-Object Masking

Implementation and Evaluation in ORB-SLAM3 and VINS-Fusion

Bolton Athit Davies

Reporting cutoff: 21 October 2026

Abstract

Class-based masking removes every detected bin, box, bucket and pallet jack from the visual front end, whether or not the object is moving. In the warehouse datasets used here, most detected objects are stationary scenery, so the filter discards good landmarks to remove a minority of harmful ones. This report documents `filter_mode:=motion`, an extension that masks a detected box only after the tracker has observed it move, its implementation in both pipelines, and its evaluation against simulator ground truth.

The result is negative. In ORB-SLAM3 the gate is at chance: pooled over 7710 ground-truth-labelled box-frames it flags 82 % of genuinely moving objects and 76 % of stationary ones, an area under the ROC curve of 0.528 where 0.5 is chance. In VINS-Fusion three successive signals were tried and all failed, one of them inverted. Both failures share a cause that is architectural rather than a matter of tuning: each estimator discards the features that disagree with its own pose estimate, so by the time a census can be taken, the evidence of motion has already been removed. The feature is implemented, documented, disabled by default, and reported here as a measured negative result.

Contents

1 Motivation **3**

1.1 What class masking does . . . 3

1.2 Why that matters in these datasets . . . 3

1.3 Relation to IQ-VIO . . . 3

2 Implementation **5**

2.1 Where the mask is consumed, and why it differs per pipeline . . . 5

2.2 Mask construction . . . 6

2.3 The census . . . 6

2.4 Concurrency and the in-flight ledger . . . 7

2.5 Verdict, association and probation . . . 7

2.6 Parameters . . . 8

2.7 Instrumentation . . . 8

2.8 Evaluation method . . . 8

3 Results **10**

3.1 Validity of the runs scored here . . . 10

3.2 ORB-SLAM3: four design iterations . . . 10

3.2.1 Run 1 — runaway . . . 10

3.2.2 Runs 2–4 — the progression . . . 10

3.3 ORB-SLAM3: the result does not survive a larger sample . . . 11

3.4 VINS-Fusion: three signals, all negative . . . 12

4 Why both pipelines fail **13**

5 Status and recommendation **14**

Chapter 1

Motivation

1.1 What class masking does

The dynamic-object filter implemented for Experiment 3 follows RY-SLAM: a YOLO detector publishes bounding boxes, and the SLAM front end refuses to use image features that fall inside them. In ORB-SLAM3 the mask is consumed inside `ORBextractor`, which discards keypoints landing on a dynamic region before the octree distributes the per-level feature quota. In VINS-Fusion it is consumed inside `FeatureTracker::setMask()`.

The filter is *semantic*: it masks objects whose *class* is one the detector was trained on. Class membership is not evidence of motion. A pallet jack parked against a wall is masked exactly like one being driven across the aisle.

1.2 Why that matters in these datasets

The proportion of detected objects that actually move is small. Measured from `/world/default/pose/info` by first-versus-last displacement (>5 cm, excluding the robot):

Table 1.1: Moving props per dataset, measured from simulator ground truth.

Dataset	World	Entities	Moving
`dataset_allsensor_realistic_000`	U static twin	131	0
`dataset_allsensor_realistic_001`	I static twin	153	0
`dataset_allsensor_realistic_002`	U dynamic	—	10
`dataset_allsensor_realistic_003`	I dynamic	—	12
`dataset_dynamic_nofloortexture_00_000`	dynamic 0	101	25

In the realistic worlds, ten to twelve props move among roughly 150 static ones. Class masking therefore spends most of its effect deleting landmarks that were never a problem. In the static twins it can only do harm: there is nothing to remove.

This is the gap motion gating is meant to close — keep the parked objects as landmarks, mask only the ones in motion.

1.3 Relation to IQ-VIO

IQ-VIO [1] addresses the same problem from the opposite direction: it keeps every feature and instead lowers the confidence of the visual pose measurement inside an EKF when the coverage of dynamic-class

detections is high. That protects the fused output but not the map, and — importantly — its interference index is computed from detection *coverage*, so like class masking it never measures motion. A parked forklift and a moving one produce the same index. The approach reported here differs in deciding per object, and in using the tracker's own geometry rather than detection area.

Chapter 2

Implementation

2.1 Where the mask is consumed, and why it differs per pipeline

The two estimators manage features differently, and the integration point follows from that difference rather than from preference.

ORB-SLAM3 — one population. Features are re-extracted from scratch every frame, so masking extraction is the whole job. The mask is consumed inside `ORBextractor`, in the loop that collects candidate keypoints before the octree distributes the per-level quota. The rule is RY-SLAM equation (1): count dynamic pixels in a 3×3 neighbourhood of the keypoint and delete it at five or more, reserve it otherwise. One threshold covers both cases the paper separates — a keypoint deep inside a dynamic region scores nine and goes, one just outside a boundary scores at most four and stays.

Two coordinate subtleties make this less trivial than it reads. At that point in the extractor a keypoint's position is in *level* coordinates relative to `(minBorderX, minBorderY)`, not image coordinates, so the offset must be added back before indexing the mask. And the neighbourhood is spaced by the level's scale factor rather than by one pixel, so a coarse-pyramid keypoint — whose true support in the source image is larger — is tested over a correspondingly larger area:

With s the level's scale factor and (x_0, y_0) the keypoint mapped back into image coordinates,

$$x_0 = (pt.x + minBorderX)\, s, \qquad y_0 = (pt.y + minBorderY)\, s,$$

$$n_{\text{dyn}} = \Big|\, \{(dx, dy) \in \{-1, 0, 1\}^2 \;:\; \text{mask}\,[y_0 + dy\, s,\; x_0 + dx\, s] = 0\} \,\Big|,$$

and the keypoint is discarded when $n_{\text{dyn}} \geq 5$. Samples falling outside the image are skipped rather than treated as dynamic.

An empty mask short-circuits the test entirely, so a run with filtering off is the stock tracker bit for bit and the recorded baseline stays reproducible.

VINS-Fusion — two populations. Features come from two places. *Tracked* points are propagated by Lucas–Kanade optical flow and are never re-detected; a point that latches onto a moving crate rides it for hundreds of frames. *New* points top the set back up to `max_cnt` wherever there is room. Masking only new detections would leave the first population untouched and the filter nearly useless.

`FeatureTracker::setMask()` sits exactly between the two. Upstream already builds a canvas there, walks the tracked points in descending track length, keeps those whose pixel reads 255, paints a `min_dist` exclusion circle at each kept point, and then hands that same `cv::Mat` to `goodFeaturesToTrack` as its mask argument. Seeding the canvas from the dynamic mask instead of a blank all-255 image therefore does both jobs with one substitution: a tracked point inside a box fails the existing test and is dropped

from `cur_pts/ids/track_cnt`, and a new point cannot be seeded there either. No new loop and no index bookkeeping — `prev_pts` is rebuilt wholesale from `cur_pts` at the end of `trackImage`, so dropping extra points breaks no correspondence.

The mask is applied to `image0` only. The right image's points are tracked *from* the already-masked left set, so nothing re-enters through the second camera.

2.2 Mask construction

Detections arrive as a `vision_msgs/Detection2DArray` stamped with the *source image's* timestamp, never `now()`; using publication time would offset every mask by the inference latency and drag the masks behind the objects. The node buffers them by that stamp and, for each frame, selects the detection nearest in time within `det_max_age`.

The mask itself is `CV_8UC1`, 255 meaning keep. Each box is inflated by `mask_dilate_px` (8) before painting, because a feature on the silhouette of a moving crate is exactly the one that corrupts the map, and detector boxes clip their objects slightly. Covered area is counted *before* painting so that overlapping boxes are not double-counted.

A safety valve discards the whole mask when it would cover more than `max_mask_fraction` (0.8) of the frame: one runaway false positive spanning the image would starve the tracker and lose the map, which is far worse than not filtering that frame. The threshold is measured rather than guessed — over all 791 frames of an earlier dynamic bag the dilated boxes covered a mean of 43.5 %, p90 61 %, maximum 72.8 %, so a limit of 0.6 would have fired during entirely normal operation.

When a frame's detection contains no boxes the node returns an *empty* `cv::Mat` rather than an all-255 one. That is both correct and cheaper: the extractor short-circuits on it instead of running the per-keypoint test.

2.3 The census

Motion gating adds a read-back path that the class filter does not need.

ORB-SLAM3. The fork's `System::TrackStereo` gains an optional `voteBoxes` argument and a `GetBoxVotes()` accessor. After tracking returns, every left keypoint of the current frame is assigned to the first box containing it, or to the background if it lies outside all of them, and two counters are incremented: keypoints, and keypoints that ended the frame as inliers.

The boxes are given in original image coordinates and must be scaled by the same factor the images were, since `TrackStereo` may rectify or resize before feeding the tracker. Overlapping boxes credit the keypoint to the first match, which keeps the partition exact — every keypoint is counted once, so the background count is a true complement.

"Inlier" is deliberately narrow. A map point qualifies only when it

1. is attached to the keypoint and not flagged an outlier by pose optimisation;
2. satisfies `Observations() > 0`, i.e. is anchored at a keyframe. ORB creates *temporary* map points from the previous frame's stereo depth on every frame; at the movers' speed those match within the projection search window and would mask the very motion being looked for;
3. is at least `motion_min_point_age` frames old (`System::SetBoxVoteMinAge`), because local mapping inserts a keyframe roughly every ten frames and re-anchors fresh points on a mover, so "keyframe-anchored" alone is only about 0.3 s of baseline.

VINS-Fusion. There is no per-frame inlier flag, so the analogue is whether the sliding window holds a solved, positive depth for the feature (`FeaturePerId::isSolved()`). The census runs on the backend thread immediately after `processImage()` — the only instant at which the window's feature list and this frame's observations are both current — and pushes its result onto a bounded queue that the node drains at the start of the next `maskFor()`.

2.4 Concurrency and the in-flight ledger

The verdict for a frame is produced on a different thread from the one that consumed its mask, so the two must be reconciled by timestamp.

In ORB-SLAM3 the mask is built in the ROS subscription callback, the frame is pushed to a single-slot queue, tracking runs on the wrapper's worker thread, and the census returns through the result callback. In VINS the mask is built in the image callback and the census is produced by the backend thread. In both cases the node keeps an *in-flight ledger*: a map from frame timestamp to the boxes handed out for that frame, together with the verdict each inherited. When a census arrives, its timestamp is looked up, the verdicts are updated, and every older entry is discarded — those frames will never be scored.

Two consequences follow, and both are properties of the design rather than bugs:

- The ledger is bounded (30 entries). If the census falls further behind than that, entries are evicted unscored and their boxes remain *unknown* permanently.
- A frame dropped before tracking never produces a census, so its ledger entry is discarded when the next census overtakes it.

All ledger access is under one mutex, and the census carries raw counts only — no policy crosses the thread boundary, which keeps the decision rule entirely inside the node and changeable without touching either estimator.

2.5 Verdict, association and probation

With $r = n_{\text{inlier}}/n_{\text{kp}}$ for a box and r_{bg} for the background of the same frame, the box is judged *moving* when $r < \alpha\, r_{\text{bg}}$ and *static* otherwise. The comparison is against the background of the *same* frame rather than an absolute threshold, because the inlier rate varies with viewpoint, texture and tracking health; the background is the per-frame control.

The vote is withheld, and the previous verdict inherited, when the box holds fewer than `motion_min_points` keypoints or when its association chain is younger than `motion_min_box_age`. The second gate exists because a one-frame census cannot distinguish *moving* from *newly revealed*: both have only young map points. Requiring a second of continuous association lets a static object accumulate old points before it is judged.

Each box of the next frame inherits the verdict of the previous frame's box with the highest intersection over union, provided it exceeds `motion_iou`; otherwise it is *unknown* and, by default, left unpainted. Only *moving* boxes are painted, so the filter is conservative: anything it has not positively convicted keeps its features.

Probation closes a loop that would otherwise be one-way. A painted box contains no keypoints, so its verdict could never be revisited and a single false positive would propagate through association indefinitely — which is exactly what happened in the first run (section 3.2.1). Every `motion_revote_frames` frames, a chain that is currently masked is left unpainted for one frame so that it is scored again. The cost is one frame of leakage per interval for a true mover; the benefit is that no verdict is permanent.

2.6 Parameters

Table 2.1: Motion-gate parameters and their calibrated defaults. Identical in both pipelines except `motion_min_point_age`, which has no VINS analogue.

Parameter	Default	Meaning
`filter_mode`	`class`	`class` masks every detected box; `motion` masks only boxes seen to move
`motion_iou`	0.3	IoU required to associate a box with the previous frame's verdict
`motion_min_points`	15	fewer keypoints in the box than this and the vote is withheld
`motion_inlier_ratio`	0.5	α: moving when the box's inlier ratio falls below this multiple of the background's
`motion_min_box_age`	30	frames a box must be associated before it is judged
`motion_min_point_age`	30	ORB only: map points younger than this do not count as census inliers
`motion_revote_frames`	15	frames between probation openings of a masked box
`motion_unknown_mask`	false	whether a never-judged box is masked
`mask_dilate_px`	8	box inflation before painting
`det_max_age`	0.15 s	oldest detection accepted for a frame
`max_mask_fraction`	0.8	discard the mask above this frame coverage

2.7 Instrumentation

Every quantity the evaluation needs is written by the run itself, so no number is inferred from unrelated logs.

- `motion_votes.csv` — one row per box per scored frame: box corners, the raw census (`n_kp`, `n_inlier` and the background pair), the inherited verdict, the final verdict and the chain age. This is the file the ground-truth scoring consumes.
- `yolo_mask.csv` — one row per frame: number of boxes painted, mask coverage, whether a detection matched, its age, whether the mask was rejected by the valve, and the `n_moving`/`n_static`/`n_unknown` split. Rows are flushed individually, because a run is normally ended with a signal that does not run destructors and an unflushed stream loses its tail.
- `run_summary.csv` / `filter_summary.csv` — `filter_mode`, voted totals, `detection_match_rate` and `filter_status` (`none`, `never_fired`, `degraded`, `ok`).
- VINS additionally separates `dynamic_points_dropped` from `mindistance_points_dropped`. Both kinds of rejection happen in the same loop and an earlier version attributed all of them to the filter, which overstated its effect; the split tests the original mask at each rejected point and is exact.

2.8 Evaluation method

`script/motion_votes_eval.py` scores one run. For each row of `motion_votes.csv` it reconstructs the camera pose at that timestamp, projects every prop's collision mesh into `cam0`, takes the 2-D extent as the truth box, matches the voted box to a prop by IoU, and labels that prop moving or static from its world-frame speed over a short window. Two traps are handled explicitly: the simulator publishes `/world/default/pose/info` with all-zero header stamps, so simulation time is rebuilt by interpolating through `/clock`; and the extrinsic matrix is parsed by hand because `cv::FileStorage` mis-reads the particular block in these configuration files.

`script/analyse_motion_gate.py` answers the prior question — whether a usable signal exists at all — by pooling every labelled box-frame across runs and sweeping both gates. It reports the area under the ROC curve and Youden's J rather than F_1, because roughly three quarters of labelled box-frames are moving, so a rule that calls everything moving scores $F_1 \approx 0.86$ while being useless. Its outputs are written to `output/compare/simulation/exp3-motion-gate/`.

Chapter 3

Results

3.1 Validity of the runs scored here

Two classes of defect were found during this work and are stated before the results, because they bound what the numbers can support.

Mask availability varies with machine load. The detector runs as a separate process competing with the tracker. On this hardware the fraction of frames that received a mask ranged from 1 % to 100 % across repeats of one bag, declining within a session as the laptop heated. Every filtered result must therefore be read beside `detection_match_rate` and `filter_status`: a low match rate means the filter did not run, not that filtering did not help. The two runs pooled in section 3.3 both carry `filter_status: degraded` (match rate 0.78 and 0.87).

A census cannot be compared across decision rules. It is computed downstream of its own masking, so changing the rule changes which features survive and therefore changes the census. Numbers from runs that used different verdict rules are not comparable, however similar their configuration.

3.2 ORB-SLAM3: four design iterations

The gate was developed over four runs on `dataset_allsensor_realistic_003` (12 moving props), each fixing the fault the previous one exposed.

3.2.1 Run 1 — runaway

With no probation, a single false *moving* verdict propagated through IoU association until motion mode had collapsed into class mode at $t \approx 105\,\mathrm{s}$: 81 % of static box-frames were masked. Separately, the census counted ORB's *temporary* map points, which are created from the previous frame's stereo depth; at the movers' speed of $0.17\,\mathrm{m\,s^{-1}}$ to $0.27\,\mathrm{m\,s^{-1}}$ the apparent motion is about 2 px per frame, well inside the projection search window, so a slow mover's temporary points matched and counted as inliers.

3.2.2 Runs 2–4 — the progression

Run 3 is the instructive one. The map-point age floor did what it was designed to do on movers — their median inlier ratio fell to a clean 0.00 — and did exactly the same to *static objects that had just come into view*, whose map points are equally young. The false-alarm rate doubled. The one-frame census cannot distinguish *moving* from *new*; the confound is scene novelty. Run 4's chain-age gate removes that confound by refusing to judge a box until it has been in view for a second, at the cost of the recall the age floor had bought.

Table 3.1: ORB-SLAM3 motion gate on `dataset_allsensor_realistic_003`. Per-vote figures are measured only on box-frames that reached a vote; the masked columns describe what the tracker actually experienced.

Run	Change	Masked box-frames		Per vote	
		moving	static	recall	false alarm
1	defaults	—	81 %	0.39	7.7 %
2	keyframe-anchored + probation	—	53 %	0.49	9.0 %
3	+ map-point age floor	—	62 %	0.62	15.4 %
4	+ box chain age (defaults)	56 %	30 %	0.43	5.0 %

At this point the gate masked 56 % of moving box-frames while sparing 70 % of the static landmarks that class mode destroys. That appeared to be a usable, if mediocre, trade.

3.3 ORB-SLAM3: the result does not survive a larger sample

The runs above rest on small per-vote samples — run 4's false-alarm figure comes from the subset of box-frames that passed both gates. To test whether the trade was real, the gate was scored on `dataset_dynamic_nofloortexture_00_000`, which carries 25 moving props, pooling both available runs and sweeping both gates so that the static class is large enough to estimate a false-alarm rate from.

Pooled over 7710 ground-truth-labelled box-frames (5833 moving, 1877 static), what the mask was built from:

Table 3.2: Pooled verdicts at the shipped thresholds, `dataset_dynamic_nofloortexture_00_000`. Source: `output/compare/simulation/exp3-motion-gate/`.

Truth	unknown	static	moving	masked
moving	4928	116	789	13.5 %
static	1561	14	302	16.1 %

The gate masks stationary objects slightly *more* often than moving ones. Sweeping both gates over the same pooled data shows this is not a threshold choice:

Table 3.3: Separability of the ORB census, pooled. $J = \text{recall} - \text{false alarm}$; AUC is $P(\text{moving ranks below static})$, so 0.5 is chance. `motion_min_points` held at 15.

`min_box_age`	coverage	n_{mov}	n_{stat}	recall	false alarm	AUC
0	56.4 %	3307	1044	0.820	0.760	0.528
3	41.5 %	2483	714	0.809	0.770	0.519
5	34.6 %	2088	582	0.793	0.790	0.506
10	23.2 %	1398	390	0.737	0.821	0.457
15	17.0 %	994	316	0.682	0.804	0.423
20	11.9 %	664	250	0.640	0.784	0.400
30	3.0 %	188	43	0.394	0.605	0.437

Recall and false alarm move together at every setting. The best separation anywhere in the sweep is $J = +0.061$ — recall 0.820 at 76.0 % false alarm. Loosening `motion_min_box_age` raises coverage from 3 % to 56 %, confirming that the chain age, not the feature count, is what starves the gate; but the extra samples do not improve separation, they dissolve it. At the shipped setting the AUC is 0.437, slightly *below* chance.

3.4 VINS-Fusion: three signals, all negative

The same policy on `dataset_dynamic_nofloortexture_00_000`, where 64 % of box-frames sit on genuinely moving objects:

Table 3.4: Census signals tried in VINS-Fusion and their outcomes.

Signal	Quantity	Outcome
solved depth	feature holds a solved, positive depth in the window	recall 0.25 at 23 % false alarm
reprojection residual	median per-box residual against the optimised window	**inverted**: movers score lower
residual availability	fraction of a box's features that have a residual	recall 0.56 at 41 % false alarm

Feature starvation was ruled out explicitly. Raising `max_cnt` from 150 to 250 had no effect — the scene cannot supply more corners at the configured spacing, and the per-box feature distribution was unchanged. Reducing `min_dist` from 30 to 15 px tripled the features per box, raising coverage from 19.6 % to 55.8 %, and the signal *vanished*: 94.7 % of features inside moving objects held solved depths, and not one of 83 voted rows fell below the threshold. More data made the result worse, which is the signature of a quantity that was never measuring the intended property.

Chapter 4

Why both pipelines fail

The three VINS signals and the ORB census share one structural property: all of them are read from estimator state that has already been cleaned of moving points.

- ORB-SLAM3's census counts map points that survived `TrackLocalMap`. Points inconsistent with the optimised pose have already been marked outliers. What remains inside a moving object's box is whatever happened to fit.
- VINS-Fusion's `outliersRejection` purges any feature whose average reprojection error exceeds 3 px, every frame. A mover's disagreeing features are gone within a frame or two, so the residual census measures survivors — which is why that signal came out inverted.
- A moving point also triangulates perfectly well. It converges to the *wrong* depth, but a wrong depth is still a solved depth, and no test on depth validity can separate the two.

Both estimators are designed to be robust to moving points. That robustness is exactly what makes them poor instruments for detecting motion: by the time either has an opinion about a feature, the features that disagreed with it have been discarded. This is not a tuning failure and no threshold on these quantities can repair it.

A signal that could work must be computed *before* rejection. Three candidates exist in the VINS front end and are presently unused: `rejectWithF()`, a RANSAC fundamental-matrix check that is written but commented out, whose outliers are by definition points violating epipolar geometry; `predictPtsInNextFrame()`, implemented but never called, which would give a predicted-versus-tracked displacement per feature; and `ptsVelocity`, a per-point image velocity already computed every frame, in which ego-motion is a smooth field and a mover a coherent local anomaly. The ORB-SLAM3 equivalent is the reprojection residual of each keypoint against the IMU-predicted pose, computed in `Tracking` before outlier marking. None of these has been tried.

Chapter 5

Status and recommendation

Status: implemented, evaluated, negative. The feature is complete and in the tree in both pipelines, disabled by default (`filter_mode: class`), with calibration tooling and ground-truth scoring. It does not detect motion in either pipeline.

Experiment 3 should therefore be reported on `class` mode, which is the RY-SLAM method the project set out to evaluate. Motion gating belongs in the conclusions as an attempted extension with a measured negative result and an identified architectural cause.

Two cautions for anyone extending this work. First, every encouraging result in this project's motion-gate history came from a small sample and did not survive a larger one: a separation measured on 19 static rows disappeared at 1044, and a VINS recall of 0.62 rested on 16 moving rows. No threshold or rate should be quoted before `motion_votes_eval.py` or `analyse_motion_gate.py` has scored it against ground truth with the sample size stated. Second, the per-vote figures in table 3.1 describe only the box-frames that reached a vote; the masked columns, not the per-vote columns, describe what the tracker experienced.

Bibliography

[1] Zhang, H., Ye, F., Lai, Y., Li, K., and Xu, J., 2023, “IQ-VIO: Adaptive Visual Inertial Odometry via Interference Quantization under Dynamic Environments,” **Intelligent Service Robotics**, Vol. 16, pp. 565–581.

[2] Chen, Y., et al., 2025, “RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments,” **Proceedings of ICVRV**.